Rabbit API Documentation 🐰

Welcome, **Prateeksha** & **Nikhil**! This document explains the current state of the Rabbit Backend API so that the Frontend can easily connect to it.

The backend now fully supports **Communities**, **Posts**, **Comments**, and features strict **College Silos**, **Trending algorithms**, and **Achievement Badges**!

Base URL

All requests should be made to: `http://localhost:5001/api`

---

1. Authentication Endpoints

These routes handle user login, signup, and verification. They are located under `/api/auth/`.

1.1 Register a New User

- **URL:** `/auth/register`
- **Method:** `POST`
- **Description:** Creates a new user account. The email *must* end with the college domain (e.g., `@college.edu.in`).
- **Body:**

```
{
  "email": "student@college.edu.in",
  "password": "SecurePassword123"
}
```

- **Response (200 OK):**

```
{
  "success": true,
  "message": "OTP sent to your email. Please verify."
}
```

1.2 Verify Email with OTP

- **URL:** `/auth/verify-email`
- **Method:** `POST`
- **Description:** Verifies the user's email using the OTP sent during registration.
- **Body:**

```
{
  "email": "student@college.edu.in",
  "otp": "123456"
}
```

- **Response (200 OK):**

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "60d0fe4f5311236168a109ca",
    "anonymousName": "Curious Rabbit #4521",
    "karma": 150,
    "badge": "Top Contributor"
  }
}
```

(Note: Save the `token` in localStorage or cookies for future requests!)

1.3 Login

- **URL:** `/auth/login`
- **Method:** `POST`
- **Description:** Logs in an existing verified user.
- **Body:**

```
{
  "email": "student@college.edu.in",
  "password": "SecurePassword123"
}
```

- **Response (200 OK):** Returns the `token` and `user` object (same as Verify Email).

1.4 Get Current User Profile (Protected)

- **URL:** `/auth/me`
- **Method:** `GET`
- **Headers Required:** `Authorization: Bearer <your_jwt_token>`
- **Description:** Fetches the currently logged-in user's details.
- **Response (200 OK):** Returns the user's details.

1.5 Password Management

- **Forgot Password (`POST /auth/forgot-password`):** Send `{ "email": "..." }` to receive an OTP.
- **Reset Password (`POST /auth/reset-password`):** Send `{ "email": "...", "otp": "...", "newPassword": "..." }` to update the password.
- **Resend OTP (`POST /auth/resend-otp`):** Send `{ "email": "..." }` if the OTP expires.

---

2. Core Features (Communities & Posts)

2.1 Communities

Create a Community (Protected)

- **URL:** `/api/communities`
- **Method:** `POST`
- **Body:** `{ "name": "masti", "description": "Just for fun!" }`

Get All Communities

- **URL:** `/api/communities`
- **Method:** `GET`
- **Description:** Returns all communities specifically belonging to the logged-in user's college network.

Join a Community (Protected)

- **URL:** `/api/communities/:id/join`
- **Method:** `POST`

2.2 Anonymous Posts

Create a Post (Protected)

- **URL:** `/api/posts`
- **Method:** `POST`
- **Body:** `{ "title": "My Post", "content": "Hello", "communityId": "<community_id>" }`

Get Posts by Community

- **URL:** `/api/posts/community/:communityId`
- **Method:** `GET`
- **Description:** Returns posts strictly filtered to the user's college domain. Note that the author will only have their `anonymousName` exposed! You can pass `?sort=new` or `?sort=trending` (default) parameters to change the sorting.

Upvote/Downvote a Post (Protected)

- **URL:** `/api/posts/:id/vote`
- **Method:** `POST`
- **Body:** `{ "type": "upvote" }` or `{ "type": "downvote" }`

2.3 Comments & Replies

Create a Comment/Reply (Protected)

- **URL:** `/api/comments`
- **Method:** `POST`
- **Body:**

```
{
  "content": "Great post!",
  "postId": "<post_id>",
  "parentCommentId": null // Set this to another comment's ID if replying!
}
```

Get Comments for a Post

- **URL:** `/api/comments/post/:postId`
- **Method:** `GET`
- **Description:** Returns a thread of comments.

Upvote/Downvote a Comment (Protected)

- **URL:** `/api/comments/:id/vote`
- **Method:** `POST`
- **Body:** `{ "type": "upvote" }` or `{ "type": "downvote" }`

2.4 Search

Search Communities and Posts

- **URL:** `/api/search?q=internship`
- **Method:** `GET`
- **Description:** Scans through all community names, descriptions, and post titles/contents strictly within the user's college network.

---

3. Health & Connection

3.1 Health Check

- **URL:** `/health`
- **Method:** `GET`
- **Description:** Check if the backend is running properly.
- **Response:**

```
{
  "success": true,
  "message": "🐰 Rabbit API is up and running!",
  "environment": "development",
  "timestamp": "..."
}
```

💻 Notes for Prateeksha (Frontend UI/UX)

- **CORS is enabled:** You can make requests directly from your React app (running on `localhost:3000` or `localhost:5173`) to `localhost:5001`.
- **Authentication:** Whenever a user successfully logs in or verifies their email, you will receive a `token`. You need to attach this token in the `Authorization` header as `Bearer <token>` for any future requests that require the user to be logged in (like `/auth/me`).
- **Error Handling:** The backend will return a `success: false` and a `message` string if something goes wrong (like "Invalid password" or "Email not verified"). You can display this `message` directly in your UI alerts.

Happy Building! 🚀